

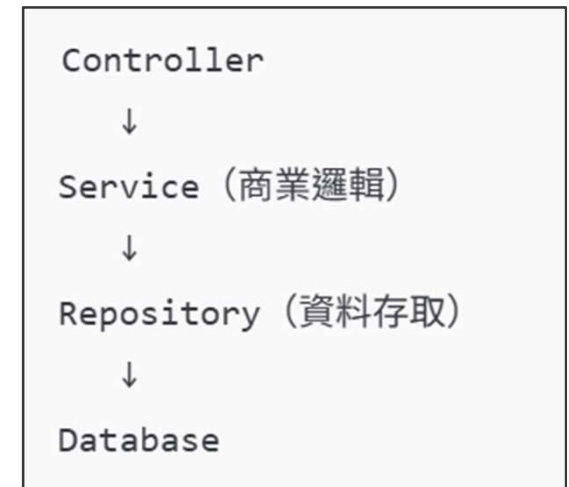
# Model-Service

- **12-1:Service 的角色**
  - Service ( 商業邏輯層 ) ..... 12-2
- **12-2:Service製作流程**
  - 定義 Service 介面 ..... 12-3
  - 實作 Service ..... 12-4
  - 註冊Service (DI繫結) ..... 12-5
  - Controller 使用 Service ..... 12-6
  - 進階註冊Service ..... 12-7
- **12-3:使用Service**
  - 有沒有 Service 的差別 ..... 12-9
  - 什麼時候「該用 Service」？ ..... 12-10
  - Service使用Repository ..... 12-11
  - Service vs Repository ..... 12-15

## Module 12

# Service ( 商業邏輯層 )

- 負責處理「業務規則」與「流程判斷」。
- 可同時呼叫一個或多個 **Repository**。
- **Controller**不直接撰寫程式邏輯與操作資料庫。
  - **Repository** 管資料。
  - **Service** 管規則。
- 透過依賴性注入(DI)實作**Service**，使維護及開發更佳有彈性，當邏輯變換時只需更新DI繫結即可，無須對**Controller**、**Model**、資料庫等程式碼做大變動。



# 定義 Service 介面

- DI的第一步：定義Interface介面。

```
public interface INotification
{
    3 個參考
    public string SendMessage(string to, string message);

    2 個參考
    public List<(string, DateTime)> GetSendListByEmail(string to);
}
```

# 實作 Service

- 實作Service：撰寫程式商業邏輯層。

```
public class EmailNotificationService:INotification
{
    2 個參考
    public string SendMessage(string to, string message)
    {
        return $"Email sent to {to} with message: {message}";
    }
    1 個參考
    public List<(string, DateTime)> GetSendListByEmail(string to)
    {
        throw new NotImplementedException();
    }
}
```

```
public class SmsNotificationService:INotification
{
    2 個參考
    public string SendMessage(string to, string message)
    {
        return $"SMS sent to {to} with message: {message}";
    }
    1 個參考
    public List<(string, DateTime)> GetSendListByEmail(string to)
    {
        throw new NotImplementedException();
    }
}
```

# 註冊Service (DI繫結)

- 在Programs.cs中實作DI繫結。

```
// Add services to the container.  
builder.Services.AddControllersWithViews();  
  
builder.Services.AddScoped<INotification, EmailNotificationService>(); // 每一次請求request都會建立新的實例  
//builder.Services.AddTransient<INotification, SmsNotificationService>(); // 每一次注入都會建立新的實例  
//builder.Services.AddSingleton<INotification, EmailNotificationService>(); // 整個應用程式生命週期只建立一個實例
```

# Controller 使用 Service

- Controller 變很乾淨：

```
private readonly INotification _notification;  
0 個參考  
public NotificationController( INotification notification)  
{  
    _notification = notification;  
}  
  
0 個參考  
public IActionResult Index()  
{  
    _notification.SendMessage("abc@abc.com", "寄給你一個email");  
    return View();  
}
```

# 進階註冊Service -1

- 建立可自由變換Service的DI註冊。

```
builder.Services.AddScoped<EmailNotificationService>();  
builder.Services.AddScoped<SmsNotificationService>();  
  
builder.Services.AddScoped<Func<string, INotification>>(serviceProvider => key =>  
{  
    return key?.ToLowerInvariant() switch  
    {  
        "email" => serviceProvider.GetRequiredService<EmailNotificationService>(),  
        "sms" => serviceProvider.GetRequiredService<SmsNotificationService>(),  
        _ => throw new KeyNotFoundException()  
    };  
});
```

## 進階註冊Service -2

- 可於程式中自由變換使用的Service。

```
public class NotificationController : Controller
{
    private readonly Func<string, INotification> _notificationFactory;
    0 個參考
    public NotificationController(Func<string, INotification> notificationFactory)
    {
        _notificationFactory = notificationFactory;
    }

    0 個參考
    public IActionResult Index()
    {
        _notificationFactory("Email").SendMessage("abc@abc.com", "寄一個email給了你喔");
        return View();
    }
}
```



# 有沒有 Service 的差別

- 沒有 Service :
  - Controller 直接操作 DbContext / Repository
  - Controller 肥大、難測試、難維護
- 有 Service
  - Controller 只管流程
  - 商業邏輯集中
  - 易測試、易擴充

# 什麼時候「該用 Service」？

- 有商業規則（價格判斷、權限、流程）。
- 一個功能需要操作多個 **Repository**。
- 中大型專案、多人協作。

# Service使用Repository -1

- 建立Service Interface :

```
public interface INorthwindShipperService
{
    2 個參考
    public IEnumerable<Shipper> GetAllShippers();
    1 個參考
    public Shipper GetShipperById(int id);
    1 個參考
    public bool CreateShipper(Shipper shipper);
    1 個參考
    public bool UpdateShipper(Shipper shipper);
    1 個參考
    public bool DeleteShipper(int id);
}
```

# Service使用Repository -2

- 建立Service並搭配先前製作的Repository使用：

```
public class NorthwindShipperService : INorthwindShipperService
{
    private readonly INorthwindShipperRepository _shipperRepo;

    0 個參考
    public NorthwindShipperService(INorthwindShipperRepository shipperRepo)
    {
        _shipperRepo = shipperRepo;
    }

    2 個參考
    public IEnumerable<Shipper> GetAllShippers()
    {
        return _shipperRepo.GetAll();
    }

    1 個參考
    public bool CreateShipper(Shipper shipper)
    {
        if (shipper != null) {
            try
            {
                shipper.CompanyName += " -CO.";
            }
        }
    }
}
```

# Service使用Repository -3

- 建立DI繫結：

```
// Add services to the container.  
builder.Services.AddControllersWithViews();  
  
builder.Services.AddDbContext<NorthwindDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("NorthwindConnectionStrings")));  
  
builder.Services.AddScoped<INorthwindShipperRepository, NorthwindShipperRepository>();  
builder.Services.AddScoped<INorthwindShipperService, NorthwindShipperService>();  
  
var app = builder.Build();
```

# Service使用Repository -3

- 使用Service :

```
public class ShippersController : Controller
{
    private readonly INorthwindShipperService _shipperService;

    0 個參考
    public ShippersController(INorthwindShipperService shipperService)
    {
        _shipperService = shipperService;
    }

    0 個參考
    public ActionResult Index()
    {
        return View(_shipperService.GetAllShippers());
    }
}
```

# Service vs Repository

- Service ( 商業邏輯/流程 ) :
  - 規則判斷 ( 能不能做 )
  - 流程協調 ( 要做哪些步驟 )
  - 跨多個 Repository 的呼叫
  - 操作交易 ( Transaction )、事件、通知等
- Repository ( 資料存取 ) :
  - 跟資料庫溝通
  - 查詢、儲存、更新
  - 只關心資料的取得與保存方式
- Service與Repository的差別在於「責任分工」，若你寫的Repository和Service長得一樣，則代表「Service 還沒有真正做Service的事」。